

Chapter 13

Linear algebra

Linear algebra is the branch of mathematics that studies vector spaces and linear transformations between them.

13.1 Vectors and vector spaces

Let's start with vectors. In the simplest form, a **vector** consists of a sequence of n values from some field (see §4.1); for most purposes, this field will be $\mathbb{R}$. The number of values (called **coordinates**) in a vector is the **dimension** of the vector. The set of all vectors over a given field of a given dimension (e.g., $\mathbb{R}^n$) forms a **vector space**, which has a more general definition that we will give later.

So the idea is that a vector represents a point in an n -dimensional space represented by its coordinates in some coordinate system. For example, if we imagine the Earth is flat, we can represent positions on the surface of the Earth as a latitude and longitude, with the point $\langle 0, 0 \rangle$ representing the origin of the system at the intersection between the equator (all points of the form $\langle 0, x \rangle$) and the prime meridian (all points of the form $\langle x, 0 \rangle$). In this system, the location of Arthur K. Watson Hall (AKW) would be $\langle 41.31337, -72.92508 \rangle$, and the location of LC 317 would be $\langle 41.30854, -72.92967 \rangle$. These are both offsets (measured in degrees) from the origin point $\langle 0, 0 \rangle$.

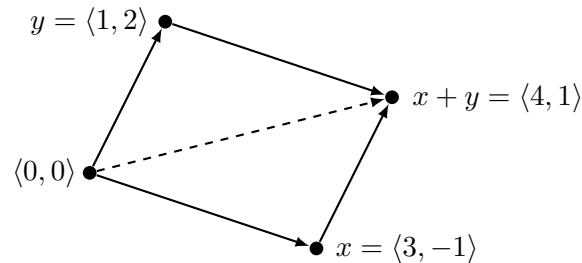


Figure 13.1: Geometric interpretation of vector addition

13.1.1 Relative positions and vector addition

What makes this a little confusing is that we will often use vectors to represent relative positions as well.¹ So if we ask the question “where do I have to go to get to LC 317 from AKW?”, one answer is to travel -0.00483 degrees in latitude and -0.00459 degrees in longitude, or, in vector terms, to follow the relative vector $\langle -0.00483, -0.00459 \rangle$. This works because we define vector addition coordinatewise: given two vectors x and y , their sum $x + y$ is defined by $(x + y)_i = x_i + y_i$ for each index i . In geometric terms, this has the effect of constructing a compound vector by laying vectors x and y end-to-end and drawing a new vector from the start of x to the end of y (see Figure 13.1.)

The correspondence between vectors as absolute positions and vectors as relative positions comes from fixing an origin 0 . If we want to specify an absolute position (like the location of AKW), we give its position relative to the origin (the intersection of the equator and the prime meridian). Similarly, the location of LC 317 can be specified by giving its position relative to the origin, which we can compute by first going to AKW ($\langle 41.31337, -72.92508 \rangle$), and then adding the offset of LC 317 from AWK ($\langle -0.00483, -0.00459 \rangle$) to this vector to get the offset directly from the origin ($\langle 41.30854, -72.92967 \rangle$).

More generally, we can add together as many vectors as we want, by adding them coordinate-by-coordinate.

This can be used to reduce the complexity of pirate-treasure instructions:

1. Yargh! Start at the olde hollow tree on Dead Man’s Isle, *if ye dare*.

¹A further complication that we will sidestep completely is that physicists will often use “vector” to mean both an absolute position and an offset from it—sort of like an edge in a graph—requiring n coordinates to represent the starting point of the vector and another n coordinates to represent the ending point. These vectors really do look like arrows at a particular position in space. Our vectors will be simpler, and always start at the origin.

2. Walk 10 paces north.
3. Walk 5 paces east.
4. Walk 20 paces south.
5. Walk $6\sqrt{2}$ paces northwest.
6. Dig 8 paces down.
7. Climb back up 6 paces. There be the treasure, argh!

In vector notation, this becomes:

1. $\langle 0, 0, 0 \rangle$
2. $+ \langle 10, 0, 0 \rangle$
3. $+ \langle 0, 5, 0 \rangle$
4. $+ \langle -20, 0, 0 \rangle$
5. $+ \langle 6, -6, 0 \rangle$
6. $+ \langle 0, 0, -8 \rangle$
7. $+ \langle 0, 0, 6 \rangle$

which sums to $\langle -4, -1, -2 \rangle$. So we can make our life easier by walking 4 paces south, 1 pace west, and digging only 2 paces down.

13.1.2 Scaling

Vectors may also be **scaled** by multiplying each of their coordinates by an element of the base field, called a **scalar**. For example, if $x = \langle -4, -1, -2 \rangle$ is the number of paces north, east, and down from the olde hollow tree to the treasure in the previous example, we can scale x by 2 to get the number of paces for Short-Legged Pete. This gives

$$2 \langle -4, -1, -2 \rangle = \langle -8, -2, -4 \rangle .$$

13.2 Abstract vector spaces

So far we have looked at vectors in $\mathbb{R}^n$, which can be added together (by adding their coordinates) and scaled (by multiplying by an element of $\mathbb{R}$). In the abstract, a **vector space** is any set that supports these operations, consistent with certain axioms that make it behave like we expect from our experience with $\mathbb{R}^n$.

Formally, a vector space consists of **vectors**, which form an additive Abelian group,² and **scalars**, which can be used to scale vectors through **scalar multiplication**. The scalars are assumed to be a field (see §4.1); the reals $\mathbb{R}$ and complex numbers $\mathbb{C}$ are typical choices.

Vector addition and scalar multiplication are related by a distributive law and some consistency requirements. When a and b are scalars, and x and y are vectors, we have

$$\begin{aligned} a(x + y) &= ax + ay \\ (a + b)x &= ax + bx \\ 0x &= 0 \\ 1x &= x \\ a(bx) &= (ab)x \end{aligned}$$

Note that in $0x = 0$, the 0 on the left-hand side is a scalar while the 0 on the right-hand side is a vector.

To avoid confusing between scalars, some writers will mark vectors using boldface ($\mathbf{x}$) or with a superimposed arrow ($\vec{x}$). Both are annoying enough to type that we will not use either convention.

It's not hard to see that the $\mathbb{R}^n$ vectors we defined earlier satisfy this definition. But there are other examples of vector spaces:

- The complex numbers $\mathbb{C}$ form a two-dimensional vector space over the real numbers $\mathbb{R}$. This is because we can represent any complex number $a + bi$ as a two-dimensional vector $\langle a, b \rangle$, and ordinary complex-number addition and multiplication by reals behaves just like vector addition and scalar multiplication in $\mathbb{R}^2$: $(a + bi) + (c + di) = (a + b) + (c + d)i$, $r(a + bi) = (ra) + (rb)i$.
- If F is a field and S is any set, then the set F^S of functions $f : S \rightarrow F$ is a vector space, where the scalars are the elements of F , $f + g$ is defined

²This means that there is an addition operation for vectors that is commutative ($x + y = y + x$), associative ($x + (y + z) = (x + y) + z$), and has an identity element 0 ($0 + x = x + 0 = x$) and inverses $-x$ ($x + (-x) = 0$).

by $(f + g)(x) = f(x) + g(x)$, and af is defined by $(af)(x) = a \cdot f(x)$. Our usual finite-dimensional real vector spaces are special cases of this, where $S = \{1, \dots, n\}$ and $F = \mathbb{R}$.

- The set of all real-valued random variables on a probability space forms a vector space, with $X + Y$ and aX defined in the usual way. For discrete probability spaces, this is another special case of F^S , since each random variable is really an element of $\mathbb{R}^\Omega$. For general probability spaces, there are some technical conditions with measurability that mean that we don't get all of $\mathbb{R}^\Omega$, but it's still the case that $X + Y$ and aX are random variables whenever X and Y are, giving us a vector space.
- In some application areas, it's common to consider restricted classes of functions with some nice properties; for example, we might look at functions from $\mathbb{R}$ to $\mathbb{R}$ that are continuous or differentiable, or infinite sequences $\mathbb{N} \rightarrow \mathbb{R}$ that converge to a finite sum. These restricted classes of functions are all vector spaces as long as $f + g$ and af are in the class whenever f and g are.

13.3 Matrices

We've seen that a **sequence** $a_1, a_2, \dots, a_n$ is really just a function from some index set ($\{1 \dots n\}$ in this case) to some codomain, where $a_i = a(i)$ for each i . What if we have two index sets? Then we have a two-dimensional structure:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix}$$

where $A_{ij} = a(i, j)$, and the domain of the function is just the cross-product of the two index sets. Such a structure is called a **matrix**. The values A_{ij} are called the **elements** or **entries** of the matrix. A sequence of elements with the same first index is called a **row** of the matrix; similarly, a sequence of elements with the same second index is called a **column**. The **dimension** of the matrix specifies the number of rows and the number of columns: the matrix above has dimension $(3, 2)$, or, less formally, it is a 3×2 matrix.³ A matrix is **square** if it has the same number of rows and columns.

Note: The convention in matrix indices is to count from 1 rather than 0. In programming language terms, matrices are written in FORTRAN.

³The convention for both indices and dimension is that rows come before columns.

Variables representing matrices are usually written with capital letters. This is to distinguish them from both scalars and vectors.

13.3.1 Interpretation

We can use a matrix any time we want to depict a function of two arguments (over small finite sets if we want it to fit on one page). A typical example (that predates the formal notion of a matrix by centuries) is a table of distances between cities or towns, such as this example from 1807:⁴

INDEX OF DISTANCES FROM TOWN TO TOWN,
In the County of Hertford.
The names of the respective Towns are on the top and side, and the square where both meet gives the distance.

	St. Alban's, . . .	<i>Distant from London, . . .</i>	<i>Miles, . . .</i>	20												
Baldock,	18	Baldock,		37												
Barnet,	10	22	Barnet,	11												
Berkhamstead, . .	12	25	18	Berkhamstead,	26											
Buntingford, . . .	24	9	24	35	Buntingford,	31										
Hatfield,	5	18	8	17	19	Hatfield,	19									
Hemel Hempstead	6	22	14	5	30	11	Hemel Hempstead,	23								
Hertford,	12	18	12	24	12	7	18	Hertford,	21							
Hoddesdon,	15	22	12	27	14	10	21	4	Hoddesdon,	17						
Puckridge,	20	13	20	31	4	15	26	8	10	Puckridge,	27					
Rickmansworth, . .	11	29	12	10	35	16	9	23	22	31	Rickmansworth,	18				
Royston,	26	8	30	35	7	26	30	20	21	11	42	Royston,	38			
Stevenage,	12	6	20	19	9	12	16	12	16	10	23	14	Stevenage,	31		
Ware,	15	15	16	27	10	10	18	3	4	6	26	17	11	Ware,	20	
Watford,	7	25	9	10	32	13	8	20	19	28	3	33	19	23	Watford,	15

Because distance matrices are *symmetric* (see below), usually only half of the matrix is actually printed.

Another example would be a matrix of counts. Suppose we have a set of destinations D and a set of origins O . For each pair $(i, j) \in D \times O$, let C_{ij} be the number of different ways to travel from j to i . For example, let origin 1 be Bass Library, origin 2 be AKW, and let destinations 1, 2, and 3 be Bass, AKW, and SML. Then there is 1 way to travel between Bass and AKW (walk), 1 way to travel from AKW to SML (walk), and 2 ways to travel from Bass to SML (walk above-ground or below-ground). If we assume that we are not allowed to stay put, there are 0 ways to go from Bass to Bass or AKW to AKW, giving the matrix

$$C = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

⁴The original image is taken from <http://www.hertfordshire-genealogy.co.uk/data/books/books-3/book-0370-cooke-1807.htm>. As an exact reproduction of a public domain document, this image is not subject to copyright in the United States.

Wherever we have counts, we can also have probabilities. Suppose we have a particle that moves between positions $1 \dots n$ by flipping a coin, and moving up with probability $\frac{1}{2}$ and down with probability $\frac{1}{2}$ (staying put if it would otherwise move past the endpoints). We can describe this process by a **transition matrix** P whose entry P_{ij} gives the probability of moving to i starting from j . For example, for $n = 4$, the transition matrix is

$$P = \begin{bmatrix} 1/2 & 1/2 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 1/2 & 1/2 \end{bmatrix}.$$

Finally, the most common use of matrices in linear algebra is to represent the coefficients of a **linear transformation**, which we will describe later.

13.3.2 Operations on matrices

Some functions of matrices are useful enough that they have names.

13.3.2.1 Transpose of a matrix

The **transpose** of a matrix A , written $A^\top$ or A' , is obtained by reversing the indices of the original matrix; $(A^\top)_{ij} = A_{ji}$ for each i and j . This has the effect of turning rows into columns and vice versa:

$$\begin{aligned} A^\top &= \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix}^\top \\ &= \begin{bmatrix} A_{11} & A_{21} & A_{31} \\ A_{12} & A_{22} & A_{32} \end{bmatrix} \end{aligned}$$

If a matrix is equal to its own transpose (i.e., if $A_{ij} = A_{ji}$ for all i and j), it is said to be **symmetric**. The transpose of an $n \times m$ matrix is an $m \times n$ matrix, so only square matrices can be symmetric.

13.3.2.2 Sum of two matrices

If we have two matrices A and B with the same dimension, we can compute their sum $A + B$ by the rule $(A + B)_{ij} = A_{ij} + B_{ij}$. Another way to say this is that matrix sums are done term-by-term: there is no interaction between entries with different indices.

For example, suppose we have the matrix of counts C above of ways of getting between two destinations on the Yale campus. Suppose that upperclassmen are allowed to also take the secret Science Hill Monorail from the sub-basement of Bass Library to the sub-basement of AKW. We can get the total number of ways an upperclassman can get from each origin to each destination by adding to C a second matrix M giving the paths involving monorail travel:

$$C + M = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 2 & 0 \\ 2 & 1 \end{bmatrix}.$$

13.3.2.3 Product of two matrices

Suppose we are not content to travel once, but have a plan once we reach our destination in D to travel again to a final destination in some set F . Just as we constructed the matrix C (or $C + M$, for monorail-using upperclassmen) counting the number of ways to go from each point in O to each point in D , we can construct a matrix Q counting the number of ways to go from each point in D to each point in F . Can we combine these two matrices to compute the number of ways to travel $O \rightarrow D \rightarrow F$?

The resulting matrix is known as the **product** QC . We can compute each entry in QC by taking a sum of products of entries in Q and C . Observe that the number of ways to get from k to i via some single intermediate point j is just $Q_{ij}C_{jk}$. To get all possible routes, we have to sum over all possible intermediate points, giving $(QC)_{ik} = \sum_j Q_{ij}C_{jk}$.

This gives the rule for multiplying matrices in general: to get $(AB)_{ik}$, sum $A_{ij}B_{jk}$ over all intermediate values j . This works only when the number of columns in A is the same as the number of rows in B (since j has to vary over the same range in both matrices), i.e., when A is an $n \times m$ matrix and B is an $m \times s$ matrix for some n, m , and s . If the dimensions of the matrices don't match up like this, the matrix product is *undefined*. If the dimensions do match, they are said to be **compatible**.

For example, let $B = (C + M)$ from the sum example and let A be the number of ways of getting from each of destinations 1 = Bass, 2 = AKW, and 3 = SML to final destinations 1 = Heaven and 2 = Hell. After consulting with appropriate representatives of the Divinity School, we determine that one can get to either Heaven or Hell from any intermediate destination in one way by dying (in a state of grace or sin, respectively), but that Bass Library provides the additional option of getting to Hell by digging. This

gives a matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 1 \end{bmatrix}.$$

We can now compute the product

$$A(C+M) = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 0 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 0 + 1 \cdot 2 + 1 \cdot 2 & 1 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 \\ 2 \cdot 0 + 1 \cdot 2 + 1 \cdot 2 & 2 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 4 & 3 \end{bmatrix}.$$

One special matrix I (for each dimension $n \times n$) has the property that $IA = A$ and $BI = B$ for all matrices A and B with compatible dimension. This matrix is known as the **identity matrix**, and is defined by the rule $I_{ii} = 1$ and $I_{ij} = 0$ for $i \neq j$. It is not hard to see that in this case $(IA)_{ij} = \sum_k I_{ik}A_{kj} = I_{ii}A_{ij} = A_{ij}$, giving $IA = A$; a similar computation shows that $BI = B$. With a little more effort (omitted here) we can show that I is the *unique* matrix with this identity property.

13.3.2.4 The inverse of a matrix

A matrix A is **invertible** if there exists a matrix A^{-1} such that $AA^{-1} = A^{-1}A = I$. This is only possible if A is square (because otherwise the dimensions don't work) and may not be possible even then. Note that it is enough to find a matrix such that $A^{-1}A = I$ to show that A is invertible.

To try to invert a matrix, we start with the pair of matrices A, I (where I is the identity matrix defined above) and multiply both sides of the pair from the left by a sequence of transformation matrices $B_1, B_2, \dots, B_k$ until $B_k B_{k-1} \cdots B_1 A = I$. At this point the right-hand matrix will be $B_k B_{k-1} \cdots B_1 = A^{-1}$. (We could just keep track of all the B_i , but it's easier to keep track of their product.)

How do we pick the B_i ? These will be matrices that (a) multiply some row by a scalar, (b) add a multiple of one row to another row, or (c) swap two rows. We'll use the first kind to make all the diagonal entries equal one, and the second kind to get zeroes in all the off-diagonal entries. The third kind will be saved for emergencies, like getting a zero on the diagonal.

That the operations (a), (b), and (c) correspond to multiplying by a matrix is provable but tedious.⁵ Given these operations, we can turn any

⁵The tedious details: To multiply row r by a , use a matrix B with $B_{ii} = 1$ when $i \neq r$, $B_{rr} = a$, and $B_{ij} = 0$ for $i \neq j$; to add a times row r to row s , use a matrix B with $B_{ii} = 1$ when $i \neq r$, $B_{rs} = a$, and $B_{ij} = 0$ for all other pairs ij ; to swap rows r and s , use a matrix B with $B_{ii} = 1$ for $i \notin \{r, s\}$, $B_{rs} = B_{sr} = 1$, and $B_{ij} = 0$ for all other pairs ij .

invertible matrix A into I by working from the top down, rescaling each row i using a type (a) operation to make $A_{ii} = 1$, then using a type (b) operation to subtract A_{ji} times row i from each row $j > i$ to zero out A_{ji} , then finally repeating the same process starting at the bottom to zero out all the entries above the diagonal. The only way this can fail is if we hit some $A_{ii} = 0$, which we can swap with a nonzero A_{ji} if one exists (using a type (c) operation). If all the rows from i on down have a zero in the i column, then the original matrix A is not invertible. This entire process is known as Gauss-Jordan elimination.

This procedure can be used to solve matrix equations: if $AX = B$, and we know A and B , we can compute X by first computing A^{-1} and then multiplying $X = A^{-1}AX = A^{-1}B$. If we are not interested in A^{-1} for its own sake, we can simplify things by substituting B for I during the Gauss-Jordan elimination procedure; at the end, it will be transformed to X .

Example Original A is on the left, I on the right.

Initial matrices:

$$\begin{bmatrix} 2 & 0 & 1 \\ 1 & 0 & 1 \\ 3 & 1 & 2 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Divide top row by 2:

$$\begin{bmatrix} 1 & 0 & 1/2 \\ 1 & 0 & 1 \\ 3 & 1 & 2 \end{bmatrix} \quad \begin{bmatrix} 1/2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Subtract top row from middle row and 3-top row from bottom row:

$$\begin{bmatrix} 1 & 0 & 1/2 \\ 0 & 0 & 1/2 \\ 0 & 1 & 1/2 \end{bmatrix} \quad \begin{bmatrix} 1/2 & 0 & 0 \\ -1/2 & 1 & 0 \\ -3/2 & 0 & 1 \end{bmatrix}$$

Swap middle and bottom rows:

$$\begin{bmatrix} 1 & 0 & 1/2 \\ 0 & 1 & 1/2 \\ 0 & 0 & 1/2 \end{bmatrix} \quad \begin{bmatrix} 1/2 & 0 & 0 \\ -3/2 & 0 & 1 \\ -1/2 & 1 & 0 \end{bmatrix}$$

Multiply bottom row by 2:

$$\begin{bmatrix} 1 & 0 & 1/2 \\ 0 & 1 & 1/2 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1/2 & 0 & 0 \\ -3/2 & 0 & 1 \\ -1 & 2 & 0 \end{bmatrix}$$

Subtract $\frac{1}{2}$ ·bottom row from top and middle rows:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & -1 & 0 \\ -1 & -1 & 1 \\ -1 & 2 & 0 \end{bmatrix}$$

and we're done. (It's probably worth multiplying the original A by the alleged A^{-1} to make sure that we didn't make a mistake.)

13.3.2.5 Scalar multiplication

Suppose we have a matrix A and some constant c . The **scalar product** cA is given by the rule $(cA)_{ij} = cA_{ij}$; in other words, we multiply (or *scale*) each entry in A by c . The quantity c in this context is called a **scalar**; the term *scalar* is also used to refer to any other single number that might happen to be floating around.

Note that if we only have scalars, we can pretend that they are 1×1 matrices; $a + b = a_{11} + b_{11}$ and $ab = a_{11}b_{11}$. But this doesn't work if we multiply a scalar by a matrix, since cA (where c is considered to be a matrix) is only defined if A has only one row. Hence the distinction between matrices and scalars.

13.3.3 Matrix identities

For the most part, matrix operations behave like scalar operations, with a few important exceptions:

1. Matrix multiplication is only defined for matrices with compatible dimensions.
2. Matrix multiplication is *not commutative*: in general, we do not expect that $AB = BA$. This is obvious when one or both of A and B is not square (one of the products is undefined because the dimensions aren't compatible), but may also be true even if A and B are both square.

For a simple example of a non-commutative pair of matrices, consider

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}.$$

On the other hand, matrix multiplication *is* associative: $A(BC) = (AB)C$. The proof is by expansion of the definition. First compute

$$\begin{aligned}(A(BC))_{ij} &= \sum_k A_{ik}(BC)_{kj} \\ &= \sum_k \sum_m A_{ik}B_{km}C_{mj}.\end{aligned}$$

Then compute

$$\begin{aligned}((AB)C)_{ij} &= \sum_m (AB)_{im}C_{mj} \\ &= \sum_m \sum_k A_{ik}B_{km}C_{mj} \\ &= \sum_k \sum_m A_{ik}B_{km}C_{mj} \\ &= (A(BC))_{ij}.\end{aligned}$$

So despite the limitations due to non-compatibility and non-commutativity, we still have:

Associative laws $A + (B + C) = (A + B) + C$ (easy), $(AB)C = A(BC)$ (see above). Also works for scalars: $c(AB) = (cA)B = A(cB)$ and $(cd)A = c(dA) = d(cA)$.

Distributive laws $A(B + C) = AB + AC$, $A(B + C) = AB + AC$. Also works for scalars: $c(A + B) = cA + cB$, $(c + d)A = cA + dA$.

Additive identity $A + 0 = 0 + A = A$, where 0 is the all-zero matrix of the same dimension as A .

Multiplicative identity $AI = A$, $IA = A$, $1A = A$, $A1 = A$, where I is the identity matrix of appropriate dimension in each case and 1 is the scalar value 1.

Inverse of a product $(AB)^{-1} = B^{-1}A^{-1}$. Proof: $(B^{-1}A^{-1})(AB) = B^{-1}(A^{-1}A)B = B^{-1}(IB) = B^{-1}B = I$, and similarly for $(AB)(B^{-1}A^{-1})$.

Transposes $(A + B)^\top = A^\top + B^\top$ (easy), $(AB)^\top = B^\top A^\top$ (a little trickier). $(A^{-1})^\top = (A^\top)^{-1}$, provided A^{-1} exists (Proof: $A^\top(A^{-1})^\top = (A^{-1}A)^\top = I^\top = I$).

Using these identities, we can do arithmetic on matrices without knowing what their actual entries are, so long as we are careful about non-commutativity. So for example we can compute

$$(A + B)^2 = (A + B)(A + B) = A^2 + AB + BA + B^2.$$

Similarly, if for square A we have

$$S = \sum_{n=0}^{\infty} A^n,$$

(where $A^0 = I$), we can solve the equation

$$S = I + AS$$

by first subtracting AS from both sides to get

$$IS - AS = I$$

then applying the distributive law:

$$(I - A)S = I$$

and finally multiplying both sides from the left by $(I - A)^{-1}$ to get

$$S = (I - A)^{-1},$$

assuming $I - A$ is invertible.

13.4 Vectors as matrices

Matrices give us an alternative representation for vectors, which allows us to extend matrix multiplication to vectors as well. We'll abuse terminology a bit by referring to a $1 \times n$ or $n \times 1$ matrix as a **vector**.

A $1 \times n$ matrix is called a **row vector** for obvious reasons; similarly, an $n \times 1$ matrix is called a **column vector**. It's a good idea when using a vector as a matrix to specify whether to interpret it as a row vector or a column vector. You can also convert between the two forms using the transpose operator $x^\top$.

Vectors defined in this way behave exactly like matrices in every respect. However, they are often written with lowercase letters to distinguish them from their taller and wider cousins. If this will cause confusion with scalars,

we can disambiguate by writing vectors with a little arrow on top: $\vec{x}$ or in boldface: $\mathbf{x}$. Often we will just hope it will be obvious from context which variables represent vectors and which represent scalars, since writing all the little arrows can take a lot of time.

When extracting individual coordinates from a vector, we omit the boring index and just write x_1, x_2 , etc. This is done for both row and column vectors, so rather than write $x_i^\top$ we can just write x_i .

Vector addition and scalar multiplication behave exactly the same for vectors-as-matrices as they did for our definition of vectors-as-sequences. This justifies treating vectors-as-matrices as just another representation for vectors.

13.4.1 Length

The **length** of a vector x , usually written as $\|x\|$ or sometimes just $|x|$, is defined as $\sqrt{\sum_i x_i^2}$. The definition follows from the Pythagorean theorem: $\|x\|^2 = \sum x_i^2$. Because the coordinates are squared, all vectors have non-negative length, and only the zero vector has length 0.

Length interacts with scalar multiplication exactly as you would expect: $\|cx\| = |c| \cdot \|x\|$. The length of the sum of two vectors depends on how they are aligned with each other, but the **triangle inequality** $\|x + y\| \leq \|x\| + \|y\|$ always holds.⁶

A special class of vectors are the **unit vectors**, those vectors x for which $\|x\| = 1$. In geometric terms, these correspond to all the points on the surface of a radius-1 sphere centered at the origin. Any vector x can be turned into a unit vector $x/\|x\|$ by dividing by its length. In two dimensions, the unit vectors are all of the form $\begin{bmatrix} \cos \theta & \sin \theta \end{bmatrix}^\top$, where by convention θ is the angle from due east measured counterclockwise; this is why traveling 9 units north-west corresponds to the vector $9 \begin{bmatrix} \cos 135^\circ & \sin 135^\circ \end{bmatrix}^\top = \begin{bmatrix} -9/\sqrt{2} & 9/\sqrt{2} \end{bmatrix}^\top$. In one dimension, the unit vectors are $\begin{bmatrix} \pm 1 \end{bmatrix}$. There are no unit vectors in zero dimensions: the unique zero-dimensional vector has length 0.

⁶These properties make $\|x\|$ an example of a **norm**. A norm is any real-valued function f on vectors that is positive for all nonzero vectors, zero for the zero vector, and satisfies the scaling property $f(cx) = |c| \cdot f(x)$ and the triangle inequality $f(x + y) \leq f(x) + f(y)$. Other examples of norms are the ℓ_1 **norm** $\|x\|_1 = \sum_i |x_i|$, the ℓ_∞ **norm** $\|x\|_\infty = \max_i |x_i|$, and the general ℓ_p **norm** $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$.

The length of a vector is also known as the ℓ_2 **norm** or **Euclidean norm**. Like the ℓ_1 norm (and even the ℓ_∞ norm if you squint at it just right), the ℓ_2 norm is a special case of the ℓ_p norm.

13.4.2 Dot products and orthogonality

Suppose we have some column vector x , and we want to know how far x sends us in a particular direction, where the direction is represented by a unit column vector e . We can compute this distance (a scalar) by taking the **dot product**

$$e \cdot x = e^\top x = \sum e_i x_i.$$

For example, if $x = \begin{bmatrix} 3 & 4 \end{bmatrix}^\top$ and $e = \begin{bmatrix} 1 & 0 \end{bmatrix}^\top$, then the dot product is

$$-e \cdot x = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = 1 \cdot 3 + 0 \cdot 4 = 3.$$

In this case we see that the $\begin{bmatrix} 1 & 0 \end{bmatrix}^\top$ vector conveniently extracts the first coordinate, which is about what we'd expect. But we can also find out how far x takes us in the $\begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}^\top$ direction: this is $\begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix} \cdot x = 7/\sqrt{2}$.

By convention, we are allowed to take the dot product of two row vectors or of a row vector times a column vector or vice versa, provided of course that the non-boring dimensions match. In each case we transpose as appropriate to end up with a scalar when we take the matrix product.

Nothing in the definition of the dot product restricts either vector to be a unit vector. If we compute $x \cdot y$ where $x = ce$ and $\|e\| = 1$, then we are effectively multiplying $e \cdot y$ by c . It follows that the dot product is proportional to the length of both of its arguments. This often is expressed in terms of the geometric formulation, memorized by vector calculus students since time immemorial:

The dot product of x and y is equal to the product of their lengths times the cosine of the angle between them.

This formulation is a little misleading, since modern geometers will often *define* the angle between two vectors x and y as $\cos^{-1} \left(\frac{x \cdot y}{\|x\| \cdot \|y\|} \right)$, but it gives a good picture of what is going on. One can also define the dot-product as the area of the parallelogram with sides x and y , with the complication that if the parallelogram is flipped upside-down we treat the area as negative. The simple version in terms of coordinates is harder to get confused about, so we'll generally stick with that.

Two vectors are **orthogonal** if their dot product is zero. In geometric terms, this occurs when either one or both vectors is the zero vector or when the angle between them is $\pm 90^\circ$ (since $\cos(\pm 90^\circ) = 0$). In other words, two non-zero vectors are orthogonal if and only if they are perpendicular to each other.

Orthogonal vectors satisfy the **Pythagorean theorem**: If $x \cdot y = 0$, then $\|x + y\|^2 = (x + y) \cdot (x + y) = x \cdot x + x \cdot y + y \cdot x + y \cdot y = x \cdot x + y \cdot y = \|x\|^2 + \|y\|^2$. It is not hard to see that the converse is also true: any pair of vectors for which $\|x + y\|^2 = \|x\|^2 + \|y\|^2$ must be orthogonal (at least in $\mathbb{R}^n$).

Orthogonality is also an important property of vectors used to define coordinate systems, as we will see below.

13.5 Linear combinations and subspaces

A **linear combination** of a set of vectors $x_1 \dots x_n$ is any vector that can be expressed as $\sum c_i x_i$ for some coefficients c_i . The **span** of the vectors, written $\langle x_1 \dots x_n \rangle$, is the set of all linear combinations of the x_i .⁷

The span of a set of vectors forms a **subspace** of the vector space, where a subspace is a set of vectors that is closed under linear combinations. This is a succinct way of saying that if x and y are in the subspace, so is $ax + by$ for any scalars a and b . We can prove this fact easily: if $x = \sum c_i x_i$ and $y = \sum d_i x_i$, then $ax + by = \sum (ac_i + bd_i) x_i$.

A set of vectors $x_1, x_2, \dots, x_n$ is **linearly independent** if there is no way to write one of the vectors as a linear combination of the others, i.e., if there is no choice of coefficients that makes some $x_i = \sum_{j \neq i} c_j x_j$. An equivalent definition is that there is no choice of coefficients c_i such that $\sum c_i x_i = 0$ and at least one c_i is nonzero (to see the equivalence, subtract x_i from both sides of the $x_i = \sum c_j x_j$ equation).

13.5.1 Bases

If a set of vectors is both (a) linearly independent, and (b) spans the entire vector space, then we call that set of vectors a **basis** of the vector space. An example of a basis is the **standard basis** consisting of the vectors $[10 \dots 00]^\top, [01 \dots 00]^\top, \dots, [00 \dots 10]^\top, [00 \dots 01]^\top$. This has the additional nice property of being made of of vectors that are all orthogonal to each other (making it an **orthogonal basis**) and of unit length (making it a **normal basis**).

A basis that is both orthogonal and normal is called **orthonormal**. We like orthonormal bases because we can recover the coefficients of some arbitrary vector v by taking dot-products. If $v = \sum a_i x_i$, then $v \cdot x_j =$

⁷Technical note: If the set of vectors $\{x_i\}$ is infinite, then we will only permit linear combinations with a finite number of nonzero coefficients. We will generally not consider vector spaces big enough for this to be an issue.

$\sum a_i(x_i \cdot x_j) = a_i$, since orthogonality means that $x_i \cdot x_j = 0$ when $i \neq j$, and normality means $x_i \cdot x_i = \|x_i\|^2 = 1$.

However, even for non-orthonormal bases it is still the case that any vector can be written as a unique linear combination of basis elements. This fact is so useful we will state it as a theorem:

Theorem 13.5.1. *If $\{x_i\}$ is a basis for some vector space V , then every vector y has a unique representation $y = a_1x_1 + a_2x_2 + \cdots + a_nx_n$.*

Proof. Suppose there is some y with more than one representation, i.e., there are sequences of coefficients a_i and b_i such that $y = a_1x_1 + a_2x_2 + \cdots + a_nx_n = b_1x_1 + b_2x_2 + \cdots + b_nx_n$. Then $0 = y - y = a_1x_1 + a_2x_2 + \cdots + a_nx_n - b_1x_1 - b_2x_2 - \cdots - b_nx_n = (a_1 - b_1)x_1 + (a_2 - b_2)x_2 + \cdots + (a_n - b_n)x_n$. But since the x_i are independent, the only way a linear combination of the x_i can equal 0 is if all coefficients are 0, i.e., if $a_i = b_i$ for all i . $\square$

Even better, we can do all of our usual vector space arithmetic in terms of the coefficients a_i . For example, if $a = \sum a_ix_i$ and $b = \sum b_ix_i$, then it can easily be verified that $a + b = \sum (a_i + b_i)x_i$ and $ca = \sum (ca_i)x_i$.

However, it may be the case that the same vector will have different representations in *different* bases. For example, in $\mathbb{R}^2$, we could have a basis $B_1 = \{(1, 0), (0, 1)\}$ and a basis $B_2 = \{(1, 0), (1, -2)\}$. Because B_1 is the standard basis, the vector $(2, 3)$ is represented as just $(2, 3)$ using basis B_1 , but it is represented as $(5/2, -3/2)$ in basis B_2 .

Both bases above have the same size. This is not an accident; if a vector space has a finite basis, then all bases have the same size. We'll state this as a theorem, too:

Theorem 13.5.2. *Let $x_1 \dots x_n$ and $y_1 \dots y_m$ be two finite bases of the same vector space V . Then $n = m$.*

Proof. Assume without loss of generality that $n \leq m$. We will show how to replace elements of the x_i basis with elements of the y_i basis to produce a new basis consisting only of $y_1 \dots y_n$. Start by considering the sequence $y_1, x_1 \dots x_n$. This sequence is not independent since y_1 can be expressed as a linear combination of the x_i (they're a basis). So from Theorem 1 there is some x_i that can be expressed as a linear combination of $y_1, x_1 \dots x_{i-1}$. Swap this x_i out to get a new sequence $y_1, x_1 \dots x_{i-1}, x_{i+1}, \dots, x_n$. This new sequence is also a basis, because (a) any z can be expressed as a linear combination of these vectors by substituting the expansion of x_i into the expansion of z in the original basis, and (b) it's independent, because if there is some nonzero linear combination that produces 0 we can substitute

the expansion of x_i to get a nonzero linear combination of the original basis that produces 0 as well. Now continue by constructing the sequence $y_2, y_1, x_1 \dots x_{i-1}, x_{i+1}, \dots, x_n$, and arguing that some $x_{i'}$ in this sequence must be expressible as a combination of earlier terms by Theorem 13.5.1 (it can't be y_1 because then y_2, y_1 is not independent), and drop this $x_{i'}$. By repeating this process we can eventually eliminate all the x_i , leaving the basis $y_n, \dots, y_1$. But then any y_k for $k > n$ would be a linear combination of this basis, so we must have $m = n$. $\square$

The size of any basis of a vector space is called the **dimension** of the space.

13.6 Linear transformations

When we multiply a column vector by a matrix, we transform the vector into a new vector. This transformation is **linear** in the sense that $A(x + y) = Ax + Ay$ and $A(cx) = cAx$; thus we call it a **linear transformation**. Conversely, any linear function f from column vectors to column vectors can be written as a matrix M such that $f(x) = Mx$. We can prove this by decomposing each x using the standard basis.

Theorem 13.6.1. *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a linear transformation. Then there is a unique $n \times m$ matrix M such that $f(x) = Mx$ for all column vectors x .*

Proof. We'll use the following trick for extracting entries of a matrix by multiplication. Let M be an $n \times m$ matrix, and let e^i be a column vector with $e_j^i = 1$ if $i = j$ and 0 otherwise.⁸ Now observe that $(e^i)^\top M e^j = \sum_k e_k^i (M e^j)_k = (M e^j)_i = \sum_k M_{ik} e_k^j = M_{ij}$. So given a particular linear f , we will now *define* M by the rule $M_{ij} = (e^i)^\top f(e^j)$. It is not hard to see that this gives $f(e^j) = M e^j$ for each basis vector j , since multiplying by $(e^i)^\top$ grabs the i -th coordinate in each case. To show that $Mx = f(x)$ for all x , decompose each x as $\sum_k c_k e^k$. Now compute $f(x) = f(\sum_k c_k e^k) = \sum_k c_k f(e^k) = \sum_k c_k M(e^k) = M(\sum_k c_k e^k) = Mx$. $\square$

⁸We are abusing notation by not being specific about how long e^i is; we will use the same expression to refer to any column vector with a 1 in the i -th row and zeros everywhere else. We are also moving what would normally be a subscript up into the superscript position to leave room for the row index—this is a pretty common trick with vectors and should not be confused with exponentiation.

13.6.1 Composition

What happens if we compose two linear transformations? We multiply the corresponding matrices:

$$(g \circ f)(x) = g(f(x)) = g(M_f x) = M_g(M_f x) = (M_g M_f)x.$$

This gives us another reason why the dimensions have to be compatible to take a matrix product: If multiplying by an $n \times m$ matrix A gives a map $g : \mathbb{R}^m \rightarrow \mathbb{R}^n$, and multiplying by a $k \times l$ matrix B gives a map $f : \mathbb{R}^l \rightarrow \mathbb{R}^k$, then the composition $g \circ f$ corresponding to AB only works if $m = k$.

13.6.2 Role of rows and columns of M in the product Mx

When we multiply a matrix and a column vector, we can think of the matrix as a sequence of row or column vectors and look at how the column vector operates on these sequences.

Let $M_{i\cdot}$ be the i -th row of the matrix (the “.” is a stand-in for the missing column index). Then we have

$$(Mx)_i = \sum_k M_{ik}x_k = M_{i\cdot} \cdot x.$$

So we can think of Mx as a vector of dot-products between the rows of M and x :

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} \langle 1, 2, 3 \rangle \cdot \langle 1, 1, 2 \rangle \\ \langle 4, 5, 6 \rangle \cdot \langle 1, 1, 2 \rangle \end{bmatrix} = \begin{bmatrix} 9 \\ 21 \end{bmatrix}.$$

Alternatively, we can work with the columns $M_{\cdot j}$ of M . Now we have

$$(Mx)_i = \sum_k M_{ik}x_k = \sum_k (M_{\cdot k})_i x_k.$$

From this we can conclude that Mx is a linear combination of columns of M : $Mx = \sum_k x_k M_{\cdot k}$. Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = 1 \begin{bmatrix} 1 \\ 4 \end{bmatrix} + 1 \begin{bmatrix} 2 \\ 5 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 4 \end{bmatrix} + \begin{bmatrix} 2 \\ 5 \end{bmatrix} + \begin{bmatrix} 7 \\ 12 \end{bmatrix} = \begin{bmatrix} 9 \\ 21 \end{bmatrix}.$$

The set $\{Mx\}$ for all x is thus equal to the span of the columns of M ; it is called the **column space** of M .

For yM , where y is a row vector, similar properties hold: we can think of yM either as a row vector of dot-products of y with columns of M or as a weighted sum of rows of M ; the proof follows immediately from the above facts about a product of a matrix and a column vector and the fact that $yM = (M^\top y^\top)^\top$. The span of the rows of M is called the **row space** of M , and equals the set $\{yM\}$ of all results of multiplying a row vector by M .

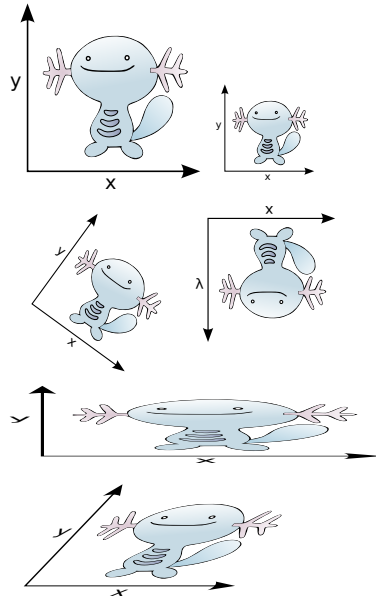
13.6.3 Geometric interpretation

Geometrically, linear transformations can be thought of as changing the basis vectors for a space: they keep the origin in the same place, move the basis vectors, and rearrange all the other vectors so that they have the same coordinates in terms of the new basis vectors. These new basis vectors are easily read off of the matrix representing the linear transformation, since they are just the columns of the matrix. So in this sense all linear transformations are transformations from some vector space to the column space of some matrix.⁹

This property makes linear transformations popular in graphics, where they can be used to represent a wide variety of transformations of images. Below is a picture of an untransformed image (top left) together with two standard basis vectors labeled x and y . In each of the other images, we have shifted the basis vectors using a linear transformation, and carried the image along with it.¹⁰

⁹The situation is slightly more complicated for infinite-dimensional vector spaces, but we will try to avoid them.

¹⁰The thing in the picture is a Pokémon known as a *Wooper*, which evolves into a *Quagsire* at level 20. This evolution is not a linear transformation.



Note that in all of these transformations, the origin stays in the same place. If you want to move an image, you need to add a vector to everything. This gives an **affine transformation**, which is any transformation that can be written as $f(x) = Ax + b$ for some matrix A and column vector b . One nifty thing about affine transformations is that—like linear transformations—they compose to produce new transformations of the same kind: $A(Cx + d) + b = (AC)x + (Ad + b)$.

Many two-dimensional linear transformations have standard names. The simplest transformation is **scaling**, where each axis is scaled by a constant, but the overall orientation of the image is preserved. In the picture above, the top right image is scaled by the same constant in both directions and the second-from-the-bottom image is scaled differently in each direction.

Recall that the product Mx corresponds to taking a weighted sum of the columns of M , with the weights supplied by the coordinates of x . So in terms of our basis vectors x and y , we can think of a linear transformation as specified by a matrix whose columns tell us what vectors to replace x and y with. In particular, a scaling transformation is represented by a matrix of the form

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix},$$

where s_x is the scale factor for the x (first) coordinate and s_y is the scale factor for the y (second) coordinate. Flips (as in the second image from the top on the right) are a special case of scaling where one or both of the scale

factors is -1.

A more complicated transformation, as shown in the bottom image, is a **shear**. Here the image is shifted by some constant amount in one coordinate as the other coordinate increases. Its matrix looks like this:

$$\begin{bmatrix} 1 & c \\ 0 & 1 \end{bmatrix}.$$

Here the x vector is preserved: $(1, 0)$ maps to the first column $(1, 0)$, but the y vector is given a new component in the x direction of c , corresponding to the shear. If we also flipped or scaled the image at the same time that we sheared it, we could represent this by putting values other than 1 on the diagonal.

For a rotation, we will need some trigonometric functions to compute the new coordinates of the axes as a function of the angle we rotate the image by. The convention is that we rotate counterclockwise: so in the figure above, the rotated image is rotated counterclockwise approximately 315° or -45° . If Θ is the angle of rotation, the rotation matrix is given by

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

For example, when $\theta = 0^\circ$, then we have $\cos \theta = 1$ and $\sin \theta = 0$, giving the identity matrix. When $\theta = 90^\circ$, then $\cos \theta = 0$ and $\sin \theta = 1$, so we rotate the x axis to the vector $(\cos \theta, \sin \theta) = (0, 1)$ and the y axis to $(-\sin \theta, \cos \theta) = (-1, 0)$. This puts the x axis pointing north where the y axis used to be, and puts the y axis pointing due west.

13.6.4 Rank and inverses

The dimension of the column space of a matrix—or, equivalently, the dimension of the range of the corresponding linear transformation—is called the **rank**. The rank of a linear transformation determines, among other things, whether it has an inverse.

Theorem 13.6.2. *If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a linear transformation with an inverse f^{-1} , then we can show all of the following:*

1. f^{-1} is also a linear transformation.
2. $n = m$, and f has **full rank**, i.e., $\text{rank}(f) = \text{rank}(f^{-1}) = m$.

- Proof.* 1. Let x and y be elements of $\text{codomain}(f)$ and let a be a scalar. Then $f(af^{-1}(x)) = a(f(f^{-1}(x))) = ax$, implying that $f^{-1}(ax) = af^{-1}(x)$. Similarly, $f(f^{-1}(x) + f^{-1}(y)) = f(f^{-1}(x)) + f(f^{-1}(y)) = x + y$, giving $f^{-1}(x + y) = f^{-1}(x) + f^{-1}(y)$. So f^{-1} is linear.
2. Suppose $n < m$. Pick any basis e^i for $\mathbb{R}^n$, and observe that $\{f(e^i)\}$ spans $\text{range}(f)$ (since we can always decompose x as $\sum a_i e^i$ to get $f(x) = \sum a_i f(e^i)$). So the dimension of $\text{range}(f)$ is at most n . If $n < m$, then $\text{range}(f)$ is a proper subset of $\mathbb{R}^m$ (otherwise it would be m -dimensional). This implies f is not surjective and thus has no inverse. Alternatively, if $m < n$, use the same argument to show that any claimed f^{-1} isn't. By the same argument, if either f or f^{-1} does not have full rank, it's not surjective.

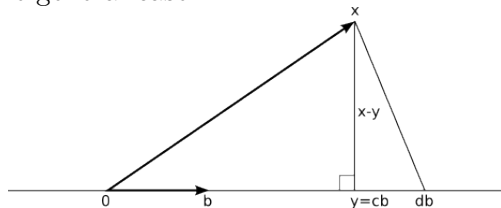
□

The converse is also true: If $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ has full rank, it has an inverse. The proof of this is to observe that if $\dim(\text{range}(f)) = n$, then $\text{range}(f) = \mathbb{R}^n$ (since $\mathbb{R}^n$ has no full-dimensional subspaces). So in particular we can take any basis $\{e^i\}$ for $\mathbb{R}^n$ and find corresponding $\{x^i\}$ such that $f(x^i) = e^i$. Now the linear transformation that maps $\sum a_i e^i$ to $\sum a_i x^i$ is an inverse for f , since $f(\sum a_i x^i) = \sum a_i f(x^i) = \sum a_i e^i$.

13.6.5 Projections

Suppose we are given a low-dimensional subspace of some high-dimensional space (e.g. a line (dimension 1) passing through a plane (dimension 2)), and we want to find the closest point in the subspace to a given point in the full space. The process of doing this is called **projection**, and essentially consists of finding some point z such that $(x - z)$ is orthogonal to any vector in the subspace.

Let's look at the case of projecting onto a line first, then consider the more general case.



A line consists of all points that are scalar multiples of some fixed vector b . Given any other vector x , we want to extract all of the parts of x that lie in the direction of b and throw everything else away. In particular, we want

to find a vector $y = cb$ for some scalar c , such that $(x - y) \cdot b = 0$. This is enough information to solve for c .

We have $(x - cb) \cdot b = 0$, so $x \cdot b = c(b \cdot b)$ or $c = (x \cdot b)/(b \cdot b)$. So the projection of x onto the subspace $\{cb \mid c \in \mathbb{R}\}$ is given by $y = b(x \cdot b)/(b \cdot b)$ or $y = b(x \cdot b)/\|b\|^2$. If b is normal (i.e. if $\|b\| = 1$), then we can leave out the denominator; this is one reason we like orthonormal bases so much.

Why is this the right choice to minimize distance? Suppose we pick some other vector db instead. Then the points x , cb , and db form a right triangle with the right angle at cb , and the distance from x to db is $\|x - db\| = \sqrt{\|x - cb\|^2 + \|cb - db\|^2} \geq \|x - cb\|$.

But now what happens if we want to project onto a larger subspace? For example, suppose we have a point x in three dimensions and we want to project it onto some plane of the form $\{c_1b_1 + c_2b_2\}$, where b_1 and b_2 span the plane. Here the natural thing to try is to send x to $y = b_1(x \cdot b_1)/\|b_1\|^2 + b_2(x \cdot b_2)/\|b_2\|^2$. We then want to argue that the vector $(x - y)$ is orthogonal to any vector of the form $c_1b_1 + c_2b_2$. As before, $(x - y)$ is orthogonal to any vector in the plane, it's orthogonal to the difference between the y we picked and some other z we didn't pick, so the right-triangle argument again shows it gives the shortest distance.

Does this work? Let's calculate: $(x - y) \cdot (c_1b_1 + c_2b_2) = x \cdot (c_1b_1 + c_2b_2) - (b_1(x \cdot b_1)/\|b_1\|^2 + b_2(x \cdot b_2)/\|b_2\|^2) \cdot (c_1b_1 + c_2b_2) = c_1(x \cdot b_1 - (b_1 \cdot b_1)(x \cdot b_1)/(b_1 \cdot b_1)) + c_2(x \cdot b_2 - (b_2 \cdot b_2)(x \cdot b_2)/(b_2 \cdot b_2)) - c_1(b_1 \cdot b_2)(x \cdot b_1)/(b_1 \cdot b_1) - c_2(b_1 \cdot b_2)(x \cdot b_2)/(b_2 \cdot b_2)$.

The first two terms cancel out very nicely, just as in the one-dimensional case, but then we are left with a nasty $(b_1 \cdot b_2)$ (much horrible junk) term at the end. *It didn't work!*

So what do we do? We could repeat our method for the one-dimensional case and solve for c_1 and c_2 directly. This is probably a pain in the neck. Or we can observe that the horrible extra term includes a $(b_1 \cdot b_2)$ factor, and if b_1 and b_2 are orthogonal, it disappears. The moral: We can project onto a 2-dimensional subspace by projecting independently onto the 1-dimensional subspace spanned by each basis vector, *provided the basis vectors are orthogonal*. And now we have another reason to like orthonormal bases.

This generalizes to subspaces of arbitrary high dimension: as long as the b_i are all orthogonal to each other, the projection of x onto the subspace $\langle b_i \rangle$ is given by $\sum b_i(x \cdot b_i)/\|b_i\|^2$. Note that we can always express this as matrix multiplication by making each row of a matrix B equal to one of the vectors $b_i/\|b_i\|^2$; the product Bx then gives the coefficients for the basis elements in the projection of x , since we have already seen that multiplying a matrix by

a column vector corresponds to taking a dot product with each row. If we want to recover the projected vector $\sum c_i b_i$ we can do so by taking advantage of the fact that multiplying a matrix by a column vector also corresponds to taking a linear combination of columns: this gives a combined operation of $B^T Bx$ which we can express as a single **projection matrix** $P = B^T B$. So projection corresponds to yet another special case of a linear transformation.

One last detail: suppose we aren't given orthonormal b_i but are instead given some arbitrary non-orthogonal non-normal basis for the subspace. Then what do we do?

The trick here is to use a technique called Gram-Schmidt orthogonalization. This constructs an orthogonal basis from an arbitrary basis by induction. At each step, we have a collection of orthogonalized vectors $b_1 \dots b_k$ and some that we haven't processed yet $a_{k+1} \dots a_m$; the induction hypothesis says that the $b_1 \dots b_k$ vectors are (a) orthogonal and (b) span the same subspace as $a_1 \dots a_k$. The base case is the empty set of basis vectors, which is trivially orthogonal and also trivially spans the subspace consisting only of the 0 vector. We add one new vector to the orthogonalized set by projecting a_{k+1} to some point c on the subspace spanned by $b_1 \dots b_k$; we then let $b_{k+1} = a_{k+1} - c$. This new vector is orthogonal to all of $b_1 \dots b_k$ by the definition of orthogonal projection, giving a new, larger orthogonal set $b_1 \dots b_{k+1}$. These vectors span the same subspace as $a_1 \dots a_{k+1}$ because we can take any vector x expressed as $\sum_{i=1}^{k+1} c_i a_i$, and rewrite it as $\sum_{i=1}^k c_i b_i + c_{k+1}(c + b_{k+1})$, and in the second term $c_{k+1}c$ reduces to a linear combination of $b_1 \dots b_k$; the converse essentially repeats this argument in reverse. It follows that when the process completes we have an orthogonal set of vectors $b_1 \dots b_m$ that span precisely the same subspace as $a_1 \dots a_m$, and we have our orthogonal basis. (But not orthonormal: if we want it to be orthonormal, we divide each b_i by $\|b_i\|$ as well.)

13.7 Further reading

Linear algebra is a key tool in graphics, scientific computing, robotics, neural networks, and many other areas of Computer Science. If you do further work in these areas, you will quickly find that we have not covered anywhere near enough linear algebra in this course. Your best strategy for remedying this deficiency may be to take an actual linear algebra course; failing that, a very approachable introductory text is *Linear Algebra and Its Applications*, by Gilbert Strang [Str05]. You can also watch an entire course of linear algebra lectures through YouTube: http://www.youtube.com/view_play_list?p=

E7DDD91010BC51F8.

Some other useful books on linear algebra:

- Golub and Van Loan, *Matrix Computations* [GVL12]. Picks up where Strang leaves off with practical issues in doing computation.
- Halmos, *Finite-Dimensional Vector Spaces* [Hal58]. Good introduction to abstract linear algebra: properties of vector spaces without jumping directly to matrices.

Matlab (which is available on the Zoo machines: type ‘matlab’ at a shell prompt) is useful for playing around with operations on matrices. There are also various non-commercial knockoffs like **Scilab** or **Octave** that are not as comprehensive as Matlab but are adequate for most purposes. Note that with any of these tools, if you find yourselves doing lots of numerical computation, it is a good idea to talk to a numerical analyst about round-off error: the floating-point numbers inside computers are not the same as real numbers, and if you aren’t careful about how you use them you can get very strange answers.